Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/TQE.2026.0000000

Noise-Resilient Quantum Convolutional Neural Networks via Bivariate Bicycle Encoding

ALEJANDRO ROSALES AND ANIMESH YADAV, (Senior Member, IEEE)

School of Electrical Engineering and Computer Science, Ohio University, Athens, OH 45701 USA (email: {ar993823, yadava}@ohio.edu)

Corresponding author: Alejandro Rosales (e-mail: ar993823@ohio.edu).

ABSTRACT Quantum neural networks (QNNs) built from parametrized quantum circuits are a candidate for processing large amounts of quantum data to reduce measurement and computational costs of detecting nonlocal quantum correlations. However, circuit-level noise during the feed-forward process perturbs their outputs and degrades classification accuracy, requiring tolerance to gate infidelities before QNNs can run on near-term quantum devices. We propose an encoded QCNN that maps an N-qubit QCNN onto the supports of the logical operators of any low-overhead, high-encoding-rate $[[n, k, d]]$ bivariate bicycle (BB) code with $k \geq N$, and we specify the logical-support mapping, encoded layer construction, and logical readout. We construct a 4-qubit QCNN and its encoding in the $[[18, 4, 4]]$ BB code, and evaluate both on symmetry-protected topological (SPT) phase recognition under circuit-level stochastic Pauli noise. With both models trained noiselessly using statevector simulations and their weights held fixed under circuit-level noise at inference, we show that the encoded QCNN reaches the same classification accuracy as the unencoded QCNN at 1.5 to 1.9 times the error rate. For near-term hardware, we further show that the qubit count of the encoded QCNN can be reduced from 18 to 11. Finally, we describe an error-corrected QCNN that extends the encoded QCNN with a coherent-feedback decoder.

INDEX TERMS Bivariate bicycle codes, noise resilience, quantum convolutional neural networks, quantum low-density parity-check codes

I. INTRODUCTION

Quantum machine learning (QML) promises to process data in exponentially large feature spaces [1], [2], but current noisy intermediate-scale quantum (NISQ) devices have limited qubit counts and high physical error rates, introduced by noise sources including stray electromagnetic fields, cosmic rays, and decoherence [3]. Quantum convolutional neural networks (QCNNs) are the quantum counterparts of classical convolutional neural networks (CNNs), applying stacked convolution and pooling layers to quantum states with a parameter count logarithmic in the number of qubits [4]. When a QCNN is trained without noise, circuit-level noise at inference shifts its outputs away from the trained decision boundary. In our simulations (Section IV), accuracy degrades even at the lowest error rate we tested.

We consider the setting in which the input state is prepared noiselessly and noise acts on the QCNN circuit itself, and obtain robustness by encoding the circuit in a quantum error-correcting code. This question is distinct from whether QCNNs offer a computational advantage. Bermejo et al. [5] showed that QCNNs are effectively classically simulable, whereas our concern is how hardware noise degrades the classification accuracy of a trained QCNN at inference, and whether encoding mitigates this degradation. It also differs from the work of Zapletal et al. [6], whose QCNNs tolerate noise in the input states by exploiting the symmetries of the phases being classified while neglecting errors in the circuit.

Quantum error correction (QEC) protects quantum information against such noise by encoding logical qubits into a subspace of a larger system of physical qubits, chosen so that errors move the state in ways that can be identified without revealing the encoded information. For over two decades, the toric code [7] has served as the canonical model for topological QEC, with thresholds high enough to enable fault-tolerant computation. However, the number of physical qubits it requires per logical qubit grows as $\mathcal{O}(d^2)$ in the code distance d. Distance-two error-detecting codes offer a lighter alternative and have been applied to variational

circuits. Gowrishankar et al. [8] encoded a variational quantum eigensolver (VQE) ansatz in the $[[4, 2, 2]]$ code, and Self et al. [9] and He et al. [10] protected parameterized and quantum approximate optimization algorithm (QAOA) circuits with its generalization, the $[[k+2, k, 2]]$ Iceberg code, on trapped-ion hardware, but these codes detect but do not correct errors as protection comes from discarding runs in which an error is flagged. In addition, He et al. found that the Iceberg code stopped improving QAOA performance as the number of logical qubits grew. Concatenating such codes yields error-correcting many-hypercube codes [11], but the weight of their syndrome measurements grows exponentially with the concatenation level [12].

Bivariate bicycle (BB) codes [13], a family of quantum low-density parity-check (qLDPC) codes, correct errors with up to an order of magnitude fewer physical qubits per logical qubit than the surface code, while keeping weight-six checks at every block size. They also have a proposed fault-tolerant architecture, the gross-code design of Yoder et al. [14], in which magic-state injection supplies the non-Clifford gates. However, that design is built on the $[[144, 12, 12]]$ gross code, which requires hundreds of physical qubits before any additional modules, too many for current devices and for circuit-level simulation of a trained QCNN. Its injection also remains costly [14], [15], and no low-overhead route has been established for running the non-Clifford, variational circuits of QCNNs on BB codes.

To this end, we study the integration of low-overhead QEC encoding into gate-based quantum neural networks, proposing an encoded QCNN that maps an N-qubit QCNN onto the logical qubits of any $[[n, k, d]]$ BB code with $k \geq N$, and specifying the logical-support mapping, encoded layer construction, and logical readout. The encoded QCNN does not perform error correction, as it has no syndrome extraction or decoder, so its noise resilience comes from the encoding alone. Unlike error-detection schemes, it also discards no runs. We encode a 4-qubit QCNN in the $[[18, 4, 4]]$ BB code [13]. We use this code because its size keeps classical simulation tractable and fits current hardware, and because distance-four BB codes have been demonstrated on a long-range-coupled superconducting processor [16]. Its logical supports cover only a subset of the code's data qubits, so the encoded QCNN runs on fewer physical qubits than the full code block. Because the encoded layers are not logical operations of the code, the encoded circuit is trained directly rather than inheriting the standard QCNN's weights.

We evaluate both models on symmetry-protected topological (SPT) phase recognition under circuit-level stochastic Pauli noise. Each is trained noiselessly from multiple random initializations, with weights and decision thresholds held fixed at inference. While matching the standard QCNN's noiseless accuracy, the encoded QCNN reaches the same accuracy under noise at a higher circuit error rate.

Building on the encoding, we also describe an error-corrected QCNN that adds a variationally trained coherent-feedback decoder to the encoded QCNN (Appendix B). Between layers, the decoder applies trainable rotations on the check ancillas and feeds them back onto the data qubits, correcting errors without reading out the syndrome. Evaluating this extension is left to future work, as is training the decoder jointly with the QCNN so that it does not treat the variational rotations as errors.

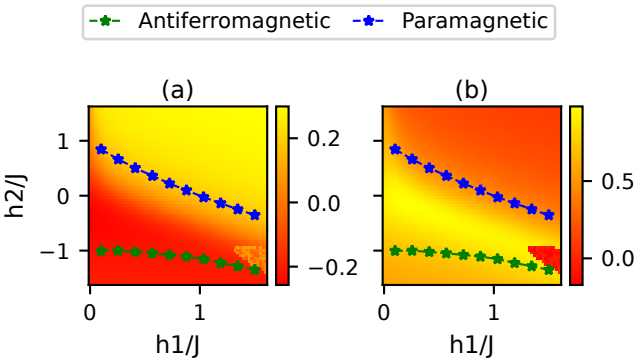


FIGURE 1: Phase recognition by the standard 4-qubit QCNN before and after training. (a) Points show a 64×64 test set of ground states over h_1 and h_2 for a Hamiltonian with $J = 1$. Phase-boundary points (green and blue stars) are the infinite-size density-matrix renormalization group (DMRG) results of [4]. Colors indicate the expectation values of the standard QCNN for $N = 4$ spins using initial, untrained weights. The sharp-edged region at the lower right ($1.32 \leq h_1/J \leq 1.6$, $-1.40 \leq h_2/J \leq -0.94$, 72 of the 4096 test states) is where the $N = 4$ ground state of (2) is exactly two-fold degenerate, so each input there is whichever state the eigensolver returns from the degenerate ground space rather than a unique ground state. It is a finite-size property of the test set, not of the QCNN weights, which is why it appears in both panels. (b) Uses the same test set with weights trained by finite differences for 100 iterations.

The rest of the paper is organized as follows. Section II presents the key background concepts used in our model implementation and the experiments conducted. Section III presents the proposed encoded QCNN. Section IV presents and discusses the results, comparing them to the standard QCNN, and Section V concludes the paper along with some directions for future research. Appendix A gives the gate-level circuit of the encoded QCNN, Appendix B describes the error-corrected QCNN extension, and Appendix B-C details the training procedure of the coherent-feedback decoder.

II. BACKGROUND

A. QUANTUM CONVOLUTIONAL NEURAL NETWORK

Classical CNNs have provided a successful ML framework for image recognition and are widely used to approximate functions due to their ability to capture hierarchical features [17], [18]. A QCNN is a quantum counterpart that can be thought of as a circuit-based ansatz of an inverse MERA with nested QEC [4].

Learning occurs through gradual tuning of its initial unitary operations [4]. The QCNN acts on input states $|\psi_\alpha\rangle \in \mathcal{H}_\mathcal{L}$, where $\alpha = 1, \ldots, M$ indexes the training samples and $\mathcal{H}_\mathcal{L} \equiv \mathcal{H}_{\mathcal{L}_0}$ denotes the full input Hilbert space before any coarse-graining,

$$\mathcal{H}_\mathcal{L} = \bigotimes_{i \in \mathcal{L}} \mathcal{H}_i, \tag{1}$$

and $\mathcal{H}_i \cong \mathbb{C}^2$ is the local qubit Hilbert space at site i.

Procedurally, the QCNN training is a supervised approximation process defined over a training set $\mathcal{T}_{\text{train}}$ and a testing set $\mathcal{T}_{\text{test}}$, where a training sample $\vec{x} \in \mathcal{T}_{\text{train}}$ is provided to the algorithm, together with its true label $\mu(\vec{x}) \in \{0, 1\}$. Here, $\mu : \mathcal{T}_{\text{train}} \cup \mathcal{T}_{\text{test}} \to \{0, 1\}$ is the underlying true map. The QCNN learns an approximate map $\tilde{\mu} : \mathcal{T}_{\text{train}} \cup \mathcal{T}_{\text{test}} \to \{0, 1\}$ by minimizing a loss function that assigns a penalty to deviations of $\tilde{\mu}(\vec{x})$ from $\mu(\vec{x})$ over the training set. Following training, the learned function $\tilde{\mu}$ is evaluated on unseen samples $\vec{x} \in \mathcal{T}_{\text{test}}$, where the goal is to minimize the generalization error $\Pr_{\vec{x} \in \mathcal{T}_{\text{test}}} [\tilde{\mu}(\vec{x}) \neq \mu(\vec{x})]$. We now define the set of steps in accordance to [4].

1) Recognizing a 1D SPT Phase

A QCNN takes advantage of the layered structure of MERA to classify the phase, where the input layer takes an unknown N-input quantum state. After the input layer, the hidden layers are the convolutional, pooling, and fully connected layers [4], similar to a CNN. The convolutional layer is given as a quasi-local unitary $U_\mathcal{L}$ and is applied in a translationally invariant manner for a finite depth. During pooling, a portion of qubits is measured, and the outcome determines the unitary rotation $V_\mathcal{L}$ applied to nearby qubits, allowing for nonlinearity through the reduction of degrees of freedom. In our implementation the measurement is deferred, so each pooled qubit coherently controls $V_\mathcal{L}$ on its neighbor (Fig. 2). The convolutional and fully connected layers apply quasi-local unitaries to the active qubits, while the pooling layer reduces the number of active qubits, so that subsequent layers act only on the qubits it retains. The number of convolution and pooling layers remains fixed throughout, while the model learns the unitaries. The convolution and pooling layers are applied a series of times until the system is sufficiently small, and a fully connected layer is applied to the remaining qubits as a unitary F. The outcome of the circuit is measured, and the prediction is obtained.

The final operator measured by the circuit recognizes the SPT phase. The set of phases considered consists of ground states $\{|\psi_{\text{GS}}\rangle\}$ corresponding to a family of Hamiltonians defined on an N-site spin-$\frac{1}{2}$ chain with open boundary conditions

$$H = -J \sum_{i=1}^{N-2} Z_i X_{i+1} Z_{i+2} - h_1 \sum_{i=1}^{N} X_i - h_2 \sum_{i=1}^{N-1} X_i X_{i+1}, \tag{2}$$

where X_i and Z_i are Pauli operators acting on site i, $J > 0$ is the three-body interaction strength, h_1 is the transverse field strength, and h_2 is the nearest-neighbor coupling strength.

The phase is detected as a nonzero expectation value of Z on the readout qubit, the qubit that remains active after the final pooling layer, denoted $\langle E \rangle$. Given an input $|\psi_\alpha\rangle \in \mathcal{P}$, where $\mathcal{P}$ denotes the set of ground states belonging to a given phase, and the QCNN circuit $\mathcal{C}_{\text{QCNN}}$ with weights Θ, the expectation value is computed from the output state vector as

$$\langle E \rangle = \sum_b \lambda(b) \left| \langle b \,|\, \mathcal{C}_{\text{QCNN}} \,|\, \psi_\alpha \rangle \right|^2, \tag{3}$$

where the sum runs over all computational basis states $|b\rangle$, and $\lambda(b) \in \{+1, -1\}$ is the Z eigenvalue of the readout qubit in $|b\rangle$.

2) Training the QCNN

We used the QCNN for quantum phase recognition (QPR) by applying it to a class of one-dimensional many-body systems. Specifically, the class considered is a $\mathbb{Z}_2 \times \mathbb{Z}_2$ phase. The $N = 4$ training and test ground states of (2) were obtained as the lowest-eigenvalue eigenvectors returned by the sparse eigensolver of SciPy, with $J = 1$. The symmetry operators generating the $\mathbb{Z}_2 \times \mathbb{Z}_2$ symmetry are

$$X_{\text{even}} = \prod_{i \in \text{even}} X_i \quad \text{and} \quad X_{\text{odd}} = \prod_{i \in \text{odd}} X_i, \tag{4}$$

where X_i is the Pauli-X operator acting on site i. The phase-boundary points in Fig. 1 are the infinite-size DMRG [19] results of [4], located from the second-order derivative of the ground-state energy density with respect to h_2 at fixed h_1.

As illustrated in Fig. 1, the QCNN classifies whether a ground state of (2) lies in the SPT phase, which contains the $S = 1$ Haldane chain [4], or in the paramagnetic or antiferromagnetic phase. The paramagnetic phase occurs when the transverse field h_1 is large relative to J, and the antiferromagnetic phase when h_2 is sufficiently negative. The antiferromagnetic phase breaks the $\mathbb{Z}_2 \times \mathbb{Z}_2$ symmetry, while the paramagnetic phase preserves it but is trivial. Along $h_1 = 0.5J$, the SPT–paramagnetic transition lies at $h_2/J = 0.423$ [4]. Along the training line $h_2 = 0$, the transition lies at $h_1 = J$.

During training, the model is given a classified training set $\mathcal{T}_{\text{train}} = \{(|\psi_\alpha\rangle, y_\alpha) : \alpha = 1, ..., M\}$, where y_α is the respective 0 or 1 binary classification for some input state $|\psi_\alpha\rangle$. The expected output of the QCNN is computed as $f_{\{U_i, V_i, F\}}(|\psi_\alpha\rangle)$ (the expected output computation is described in Section III-D). The cost function of the model is computed using the mean-squared error (MSE) of (5)

$$\texttt{MSE} = \frac{1}{M} \sum_{\alpha=1}^{M} \left(y_\alpha - f_{\{U_i, V_i, F\}}(|\psi_\alpha\rangle) \right)^2. \tag{5}$$

In the formulation of [4], the pooling layer measures a subset of qubits and conditions rotations on the outcomes, lowering the degrees of freedom for feature extraction. Our implementation defers these measurements (Section II-A1), so the circuit is unitary, and we follow the training procedure of [4].

The model learns by iteratively optimizing all initially assigned unitaries until convergence, using the finite-difference method of [4] to minimize the MSE cost function in (5). To calculate the gradient $\nabla_\Theta \mathrm{MSE}$ of the MSE with respect to the parameterized weights Θ, each parameter $\theta_j \in \Theta$ in the QCNN circuit is shifted by $\pm\epsilon$, with $\epsilon = 10^{-4}$. The weights are then updated via gradient descent as $\Theta \leftarrow \Theta - \eta \nabla_\Theta \mathrm{MSE}$, where η is the learning rate, initialized to 1. The learning rate is updated after every iteration by the bold-driver rule [20], where η is multiplied by 1.05 when the training loss decreases and halved otherwise.

Each QCNN is trained on 40 spin-$\frac{1}{2}$ ground states at evenly spaced values of $h_1 \in [0, 1.6]$ with $h_2 = 0$, labeled SPT when $h_1 \leq 1$, for 100 iterations under noiseless statevector simulation. Both the standard QCNN and the encoded QCNN of Section III-C are trained from 10 independent random parameter initializations. For the standard QCNN, these yield 10 distinct trained models, and reported results are means over them, so its error bars reflect variation across both trainings and noise realizations. For the encoded QCNN, all 10 initializations converge to the same classifier up to an overall scale, indicating that its training is insensitive to initialization. Therefore, results describe a single trained classifier, and since its runs agree up to scale without noise, we attribute its error bars to variation over noise realizations. A decision threshold is fit separately for each trained run, chosen to maximize accuracy on the noiseless training set with candidates at the midpoints between consecutive sorted training outputs. We refer to this value as the fitted threshold. It is obtained in the same way for every model, from that model's noiseless training outputs alone. Evaluation of phase recognition performance by the model is done by classifying a 64×64 test set of 4096 ground states on evenly spaced grids of $h_1 \in [0, 1.6]$ and $h_2 \in [-1.6, 1.6]$, using the fixed trained weights.

B. BIVARIATE BICYCLE CODES

The BB code is a type of Calderbank-Shor-Steane (CSS) stabilizer code that leverages bivariate polynomials over the quotient ring $R = \mathbb{F}_2[x, y]/(x^\ell - 1, y^m - 1)$, where ℓ and m are positive integers, and $\mathbb{F}_2 = \{0, 1\}$ is the binary field. An $[[n, k, d]]$ BB code $\mathrm{QC}(A, B)$ is a CSS code defined by two polynomials $A, B \in R$, where $\mathrm{QC}(A, B)$ denotes the associated quasi-cyclic code generated by the pair (A, B) [21]. Each polynomial is realized as an $\ell m \times \ell m$ binary matrix by the ring homomorphism sending $x \mapsto S_\ell \otimes I_m$ and $y \mapsto I_\ell \otimes S_m$, where S_r is the $r \times r$ cyclic shift matrix and I_r the $r \times r$ identity. Writing A and B for the resulting matrices, the X-type and Z-type parity-check matrices for a BB code denoted $\mathrm{QC}(A, B)$ are

$$\mathbf{H}^X = [A \mid B] \quad \text{and} \quad \mathbf{H}^Z = [B^T \mid A^T], \tag{6}$$

where the vertical bar indicates stacking matrices horizontally, T denotes matrix transpose, and each of size $\ell m \times 2\ell m$. The code then has parameters

$$\begin{aligned} n &= 2\ell m, \\ k &= 2 \cdot \dim\left(\ker(A) \cap \ker(B)\right), \\ d &= \min\left\{|v| : v \in \ker(\mathbf{H}^X) \setminus \mathrm{rs}(\mathbf{H}^Z)\right\}, \end{aligned} \tag{7}$$

where $\mathbf{H}$ denotes a binary matrix, $\ker(\mathbf{H})$ denotes the set of vectors orthogonal to each row of $\mathbf{H}$, $\mathrm{rs}(\mathbf{H})$ is the linear span of its rows, and $|v| = |\{i : v_i \neq 0\}|$ is the Hamming weight of $v \in \mathbb{F}_2^n$. The minimization in (7) runs over Z-type logical representatives and therefore returns d_Z. The matrices $\mathbf{H}^X$ and $\mathbf{H}^Z$ in (6) are exchanged by transposition together with a swap of the two blocks. The construction is therefore symmetric under $X \leftrightarrow Z$, and $d_X = d_Z \equiv d$.

III. PROPOSED BIVARIATE-BICYCLE CODED QCNN

In this section, we define the parameters and construction of the $[[18, 4, 4]]$ BB code. Thereafter, we describe the architecture and process for encoding the QCNN input states, encoding the QCNN circuit into the architecture of the BB code for noise mitigation, and decoding the logical readout of the encoded circuit.

A. QUBIT MAPPING

We use a BB code with parameters $[[18, 4, 4]]$, constructed with $\ell = 3$ and $m = 3$, with polynomials $A = 1 + x + y$ and $B = 1 + x^2 + y^2$. We compute the check matrices $\mathbf{H}^X$ and $\mathbf{H}^Z$ which encode $k = 4$ logical qubits with distance $d = 4$. The corresponding circuit follows the architecture described in Section II-B.

Let $\mathbf{L}_Z \in \mathbb{F}_2^{k \times n}$ denote the logical-Z operator matrix of the BB code, computed using the construction of [13], where row i is the support of the logical operator $\bar{Z}_i$ on the $n = 18$ data qubits. The mapping ζ assigns to logical qubit i the support $\mathcal{P}_i$ of row i of $\mathbf{L}_Z$, giving

$$\begin{aligned} \mathcal{P}_0 &= \{1, 2, 3, 5, 6, 7\}, & \mathcal{P}_1 &= \{0, 1, 4, 5, 6, 8\}, \\ \mathcal{P}_2 &= \{0, 2, 6, 9\}, & \mathcal{P}_3 &= \{0, 2, 3, 5, 6, 10\}. \end{aligned} \tag{8}$$

Because this BB code instance is self-dual, $\mathbf{L}_X$ yields the same mapping ζ.

Under ζ the four logical supports of (8) are supported on only 11 distinct physical qubits. These qubits are $\{0, \ldots, 10\}$ of the 18 data qubits, with $\{0, 1, 2, 3, 5, 6\}$ shared across more than one logical qubit and $\{4, 7, 8, 9, 10\}$ unique to a single logical qubit. Index 6 lies in the support of all four logical qubits. The remaining seven data qubits carry no logical support under ζ and are acted upon by no physical gate of the encoded circuit. Thus, the two 9-qubit data blocks of the code, indexed $\{0, \ldots, 8\}$ and $\{9, \ldots, 17\}$ and corresponding to the two halves of $\mathbf{H}^X = [A \mid B]$, collapse into a single 11-qubit register $\mathcal{D}$ comprising all of the first block and qubits 9 and 10 of the second. For simulations that retain the 18 check qubits of the code, this reduces the system from 36 to 29 qubits. The encoded QCNN has no check registers and runs on the 11 qubits of $\mathcal{D}$ alone (Section III-B). We emphasize that the 18 of the code parameters and the 11 of the simulated

register are distinct quantities. The former counts the code's data qubits, and the latter counts the subset carrying logical support under this particular choice of L_Z basis.

Each single-qubit gate $G \in \mathcal{C}$ acting on logical qubit q_i is replaced by the fan-out operation

$$G^{\otimes|\mathcal{P}_i|} = \bigotimes_{j\in\mathcal{P}_i} G_j, \tag{9}$$

where $\mathcal{P}_i \subseteq \{0,1,\ldots,n-1\}$ is the set of physical qubit indices corresponding to the ith logical qubit under the code's qubit mapping $\zeta : q_i \mapsto \mathcal{P}_i$.

Each CNOT between control qubit q_c and target qubit q_t is implemented as pairwise fan-out mapping

$$\mathrm{CNOT}^{\otimes} = \prod_{\substack{r=1 \\ c_r\neq t_r}}^{\min(|\mathcal{P}_c|,|\mathcal{P}_t|)} \mathrm{CNOT}_{c_r\to t_r}, \tag{10}$$

where c_r and t_r denote the rth smallest indices of $\mathcal{P}_c$ and $\mathcal{P}_t$, so the two supports are paired index by index in ascending order. The factors are applied in increasing order of r, since they need not commute when the two supports share qubits. Indices beyond the length of the shorter support are left unpaired, so a CNOT involving $\mathcal{P}_2$ fans out to at most four physical CNOTs, and pairs with $c_r = t_r$ are omitted. Gates involving stabilizer check qubits or reset operations are passed through without the fan-out operations, as they are already defined at the physical level in the encoded circuit. For a circuit of depth δ containing g two-qubit gates, the encoded circuit has depth at most $\mathcal{O}(\delta)$ and gate count at most $\mathcal{O}(g \cdot n)$.

Because all four logical qubits share a single $[[18,4,4]]$ block, pairing their supports index by index is not in general a logical operation of the code, and we return to this intra-block limitation in Section III-E.

We tested the encoded gate set directly. Each of the 2^4 logical basis states was prepared with physical X gates placed by ζ of (8), passed through the encoded gate, and decoded using the procedure of Section III-D. Correct encoding returns the prepared state for single-qubit gates and the CNOT image for two-qubit gates. For Clifford gates, all four single-qubit fan-out operations reproduced the intended logical action on all 16 basis states, whereas the two-qubit layers did not, as expected from the intra-block limitation.

B. ENCODED INPUT

The QCNN input is a 4-spin ground state of the cluster-Ising Hamiltonian, drawn from any of its phases, $|\psi\rangle = \sum_b \alpha_b |b\rangle$, a state on $(\mathbb{C}^2)^{\otimes 4}$ whose basis vectors are indexed by $b \in \mathbb{F}_2^4$. The index map E is obtained from ζ: writing $\chi(\mathcal{P}_i) \in \mathbb{F}_2^{11}$ for the indicator vector of the support $\mathcal{P}_i$ of (8),

$$E : \mathbb{F}_2^4 \to \mathbb{F}_2^{11}, \qquad E(b) \;=\; \bigoplus_{i=0}^{3} b_i\, \chi(\mathcal{P}_i), \tag{11}$$

where the sum is modulo 2 rather than a set union. Each shared index in $\{0,1,2,3,5,6\}$ is therefore flipped exactly when an odd number of the logical bits whose supports contain it are set. In particular, index 6, which lies in all four supports, is flipped exactly when $|b|$ is odd. Since the rows of $\mathbf{L}_Z$ are linearly independent, E is injective, so the 16 encoded basis states are mutually orthogonal, $V_\zeta^\dagger V_\zeta = I_{16}$ and the encoding is the linear isometry

$$V_\zeta : (\mathbb{C}^2)^{\otimes 4} \to (\mathbb{C}^2)^{\otimes 11}, \qquad V_\zeta|b\rangle \;=\; |E(b)\rangle, \tag{12}$$

fixed by its action on the computational basis and extended by linearity, so that the amplitudes of $|\psi\rangle$ are carried over unchanged. The encoded input is

$$|\overline{\psi}\rangle \;=\; V_\zeta|\psi\rangle \;=\; \sum_{b\in\mathbb{F}_2^4} \alpha_b\, |E(b)\rangle_{\mathcal{D}}. \tag{13}$$

Each $V_\zeta|b\rangle$ is the state obtained by applying the single-qubit fan-out (9) with $G = X$ to $|0\rangle^{\otimes 11}$ for every i with $b_i = 1$, so state preparation and the encoded gate set use the same construction. The encoded input is in general not a product state, since the input ground state is entangled and (13) carries that entanglement into $\mathcal{D}$. In the encoded QCNN $\overline{\mathcal{C}}_{\mathrm{QCNN}}$ the check registers are absent, and the simulated state is (13) alone on 11 qubits.

C. CONSTRUCTION OF THE ENCODED QCNN

We refer to our proposed noise-resilient architecture, which encodes the QCNN without syndrome extraction or correction, as the encoded QCNN, denoted $\overline{\mathcal{C}}_{\mathrm{QCNN}}$ which acts on the encoded input $|\overline{\psi}\rangle = V_\zeta|\psi\rangle$ given in (13). The encoded state occupies the 11-qubit register $\mathcal{D}$ rather than the code's full 18 data qubits, because only the 11 indices in $\bigcup_i \mathcal{P}_i$ carry logical support under ζ (Section III-A). Within each layer, the single-qubit gates use the fan-out construction of (9) and the two-qubit gates use the fan-out construction of (10), so that $\overline{\mathcal{C}}_{\mathrm{QCNN}}$ is obtained gate-by-gate from $\mathcal{C}_{\mathrm{QCNN}}$ under ζ. Fig. 2 shows the qubits of $\mathcal{D}$ acted on by each encoded gate of the QCNN under these fan-out constructions, with the gate-level circuit of $\overline{\mathcal{C}}_{\mathrm{QCNN}}$ given in Appendix A, Fig. 5.

D. LOGICAL READOUT

Let $\mathcal{E}_\zeta(\rho) = V_\zeta \rho V_\zeta^\dagger$ denote the encoding channel and $\mathcal{R}_\zeta$ the readout map of (15), which assigns a logical label to every computational basis state of $\mathcal{D}$ and reduces to the inverse of $\mathcal{E}_\zeta$ on $\mathrm{im}\, V_\zeta$. The classifier output is the expectation of $\bar{Z}$ on the measured logical qubit q^*. Let $\mathcal{A} = (a_0,\ldots,a_{|\mathcal{A}|-1})$ be the logical qubits that remain active after the final pooling layer, in ascending order, and let $q^* = a_{\lfloor|\mathcal{A}|/2\rfloor}$. The classifier output is the logical counterpart of (3). With $\lambda(\mathbf{v}) = (-1)^{v_{q^*}}$ as in (3),

$$\langle\bar{E}\rangle \;=\; \sum_{\mathbf{v}\in\mathbb{F}_2^N} \lambda(\mathbf{v})\,\hat{p}(\mathbf{v}) \;=\; 1 - 2\sum_{\mathbf{v}\,:\,v_{q^*}=1} \hat{p}(\mathbf{v}), \tag{14}$$

is the estimate of $\langle\bar{Z}_{q^*}\rangle$. Equation (14) has the form of (3), with the physical distribution $|\langle b|\mathcal{C}_{\mathrm{QCNN}}|\psi\rangle|^2$ over the 2^N basis states of the unencoded register replaced by the logical distribution $\hat{p}$, each entry of which aggregates the basis states

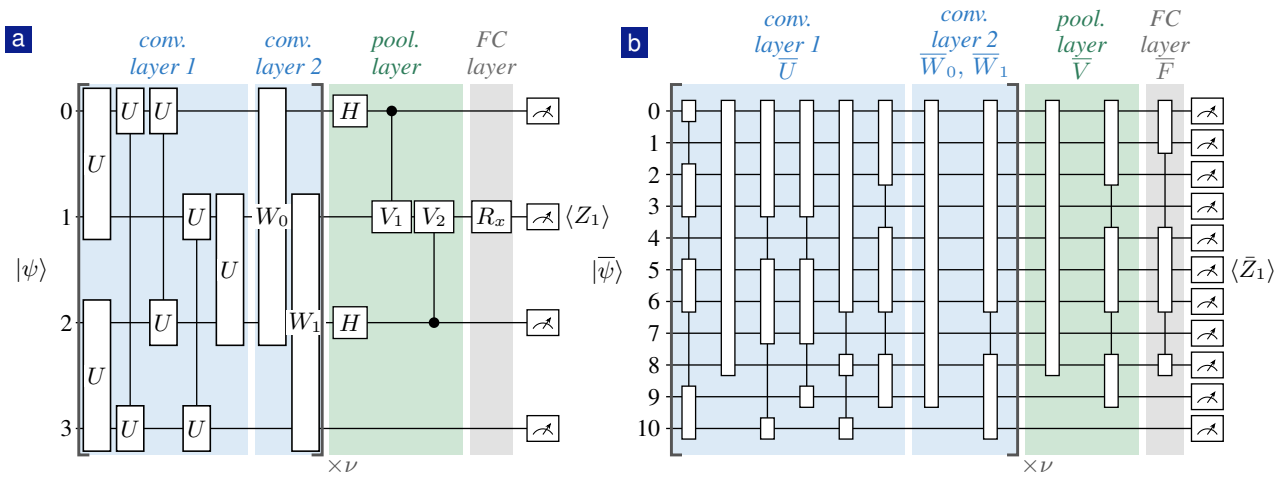


FIGURE 2: The standard 4-qubit QCNN $\mathcal{C}_{\mathrm{QCNN}}$ and its encoding $\overline{\mathcal{C}}_{\mathrm{QCNN}}$ under the mapping ζ of (8). (a) The standard QCNN acting on the input $|\psi\rangle$. The first convolution layer applies a two-qubit gate U to every pair of qubits, and the second applies the three-qubit gate W_0 to qubits $(0, 1, 2)$ and then W_1 to qubits $(1, 2, 3)$, indexed by their offset. For $N = 4$, the offset-2 gate W_2 has no complete group of three qubits and applies no gates. In the pooling layer, H is applied to qubits 0 and 2, which then control V_1 and V_2 on qubit 1. Qubits 0, 2, and 3 are not acted on further. The fully connected layer applies R_x to qubit 1, and the output is $\langle Z_1\rangle$. The bracketed convolution layers may be repeated ν times before pooling, and our implementation uses $\nu = 1$. (b) The encoded QCNN on the 11-qubit register $\mathcal{D}$, acting on the encoded input $V_\zeta|\psi\rangle$ of (13). Each column is one encoded gate of the type named in its layer header ($\overline{U}$, $\overline{W}_0$, $\overline{W}_1$, $\overline{V}$, or $\overline{F}$), and its bars mark the qubits of $\mathcal{D}$ it acts on, which is the union of the supports, $\mathcal{P}_i$ of the logical qubits the gate acts on. For example, the first column is $\overline{U}_{3,2}$. The logical readout $\mathcal{R}_\zeta$ of Section III-D is applied to the final state of $\mathcal{D}$ and returns $\langle \bar{Z}_1\rangle$.

of $\mathcal{D}$ that $\mathcal{R}_\zeta$ maps to that label. For the four-qubit QCNN considered here, $q^* = 1$.

Readout implements $\mathcal{R}_\zeta$ on the state of the data register $\mathcal{D}$. Iterating over all 2^{11} computational basis states of $\mathcal{D}$, each is zero-padded from the 11 collapsed indices to the 18 code indices to give $\mathbf{b}_j \in \mathbb{F}_2^{18}$, indexed so that physical qubit j corresponds to bit j, and assigned the logical label

$$\mathbf{v}(\mathbf{b}_j) \;=\; \left[\left(J_4\, L_X\, J_{18}\, \mathbf{b}_j\right) \bmod 2\right] \in \mathbb{F}_2^4, \qquad (15)$$

where J_r denotes the $r \times r$ exchange matrix that reverses index order. Since L_X and L_Z are not returned as a symplectically dual pair, L_X alone does not invert the encoding. Therefore, both index orders are reversed, $J_4 L_X J_{18} L_Z^{\mathsf{T}} = I_4$, so that the fan-out of X on $\mathcal{P}_q$ acts as logical X_q, consistent with (11). Each logical bit v_q of (15) depends on a single physical qubit of $\mathcal{D}$, namely qubits 7, 8, 9, and 10 for $q = 0, 1, 2, 3$, each of which lies only in the support $\mathcal{P}_q$. The probability $|\alpha_j|^2$ of each basis state is accumulated into the entry of the logical distribution indexed by $\mathbf{v}(\mathbf{b}_j)$,

$$\hat{p}(\mathbf{v}) \;=\; \sum_{j\,:\,\mathbf{v}(\mathbf{b}_j)=\mathbf{v}} |\alpha_j|^2, \qquad (16)$$

which is passed to the prediction stage. In the noiseless case $\mathbf{v} \circ E = \mathrm{id}$, so applied directly to an encoded input $V_\zeta|\psi\rangle$, $\hat{p}$ recovers $|\alpha_b|^2$ exactly.

E. LIMITATIONS

The encoded input is not a state of the $[[18, 4, 4]]$ code space. Since the rows of $\mathbf{L}_Z$ commute with the X-type checks and $\mathbf{H}^X = \mathbf{H}^Z$, $|E(b)\rangle$ satisfies every Z-type check, but any X-type check X^h maps it to $|E(b) \oplus h\rangle$, outside the image of E. The codeword would superpose $|E(b)\rangle$ over its coset $\{E(b) \oplus s : s \in \mathrm{row}(\mathbf{H}^X)\}$. Preparing a code-space input would not help here, since the layers are not logical operations of the code (Section III-A) and the state leaves the code space at the first layer. Because the encoded layers do not implement the logical QCNN gates, the variational parameters are trained on the encoded circuit $\overline{\mathcal{C}}_{\mathrm{QCNN}}$ itself, rather than trained on $\mathcal{C}_{\mathrm{QCNN}}$ and transferred to the encoding. Transferring the trained weights directly is unreliable. Loaded into the encoded circuit, the weights of all 10 trained standard QCNNs label every test state as non-SPT at their own decision thresholds, giving the majority-class accuracy of 0.591. Refitting the threshold on the encoded circuit's noiseless training outputs recovers a test accuracy of 0.813 to 0.816 for five of the models, while the other five fall to 0.29, below the majority-class rate.

We restrict our simulations to the $[[18, 4, 4]]$ BB code because the continuous rotations in the QCNN preclude efficient stabilizer simulation, and the cost of full state-vector simulation grows exponentially with the number of physical qubits. This setting carries two limitations. First, for any BB code [13], the continuous variational parameters yield rotations that are in general non-Clifford and cannot be implemented transversally in the BB data block, consistent with the Eastin–Knill theorem [22]. These non-Clifford rotations instead require resources such as magic-state injection and

distillation [14], [15], [23]. Second, the four logical qubits of the $[[18, 4, 4]]$ code share a single code block, so two-qubit logical gates between them cannot be applied transversally. Such intra-block logical Clifford gates can be realized with fold-transversal gates [24], [25], Tanner-graph automorphism gates [26], or code surgery [27], as constructed for the $[[144, 12, 12]]$ gross code [14]. Fault-tolerant execution of the QCNN would therefore incur substantial overhead in circuit depth and ancilla qubits, which we leave to future work.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

A. SIMULATION SETUP

We implement a stochastic circuit-level Pauli noise model [13], in which every gate location is subject to the same error probability p, applied independently. Single-qubit gates are preceded, with probability p, by a Pauli error drawn uniformly from $\{X, Y, Z\}$. Two-qubit CNOT gates are followed, with probability p, by an error drawn uniformly from the 15 non-identity two-qubit Paulis $\{I, X, Y, Z\}^{\otimes 2} \setminus \{II\}$. Idle errors are inserted to maintain a consistent time boundary across all qubits. In each layer, every qubit not involved in an active gate independently receives a Pauli error drawn uniformly from $\{X, Y, Z\}$ with probability p. Each error location therefore realizes a symmetric depolarizing channel, which we simulate by sampling Pauli trajectories. We sweep $p \in \{0, 0.0001, 0.0005, 0.001, 0.002, 0.003, 0.005, 0.01\}$, spanning error rates representative of current and near-term hardware, and use this model as a baseline for evaluating the noise tolerance of our BB-code-encoded QCNN against gate-level noise. All noise simulations use the same seeded random number generator for reproducibility, and each test state is evaluated on an independently sampled noisy circuit. State-preparation errors are excluded, and the classifier output is computed from the exact final statevector of each noisy circuit, so the reported results contain no measurement operations and therefore no measurement errors.

Following [4], the second convolution layer applies three-qubit gates to consecutive groups of three qubits at offsets 0, 1, and 2, each offset with its own 63 parameters, so that for larger N every three adjacent qubits are acted on by a gate. For $N = 4$, the groups at offsets 0 and 1 are qubits $(0, 1, 2)$ and $(1, 2, 3)$, while the offset-2 layer contains no complete group of three qubits and applies no gates. Its 63 parameters are allocated but never enter the circuit, so their finite-difference gradients are zero and they keep their initial values. Of the 211 allocated parameters, both models therefore train the same 148 effective parameters, with 15 for the two-qubit convolution gate U, which is shared by all six qubit pairs, 63 for each of the two applied three-qubit gates, 6 for the pooling rotations V_1 and V_2, and 1 for the fully connected rotation F. In the encoded QCNN, every physical copy of a fanned-out gate shares the parameters of its logical gate. The standard QCNN compiles to 97 single-qubit gates and 58 CNOTs at depth 93 on 4 qubits. The encoded QCNN compiles to 530 single-qubit gates and 173 CNOTs at depth 132 on the 11 qubits of $\mathcal{D}$, after the index-wise pairing of (10) omits 105 pairs where $j = k$. Because every gate and every idle qubit in each layer is a noise location under the model above, the encoded QCNN has approximately 4.5 times as many gate locations per pass as the standard QCNN (703 against 155), and approximately 4.1 times as many locations when idle qubits are included (1279 against 314).

Only a subset of these locations can affect the output. On $\mathcal{D}$, the logical readout of Section III-D reduces to Z on data qubit 8, and under the index-wise pairing of (10) the only qubits that exchange gates with qubit 8, directly or through intermediate gates, are qubits 7 and 10, so the encoded output depends only on qubits $\{7, 8, 10\}$. Injecting faults on every location outside these three qubits leaves the encoded output unchanged. The encoded QCNN therefore has 213 fault locations that can reach its output (70 single-qubit, 23 CNOT, and 120 idle), against 285 for the standard QCNN (96 single-qubit, 58 CNOT, and 131 idle).

The standard QCNN and the encoded QCNN are each trained for 100 iterations under noiseless statevector simulation from 10 independent random initializations (Section II-A2), and the trained weights and decision thresholds are then held fixed while p is swept, so no training is performed under noise. Consequently, neither model learns to adapt to the noise, as neither noise-aware training nor per-p refitting of the decision thresholds is employed. Any improvement in robustness exhibited by the encoded QCNN relative to the standard QCNN therefore arises from the BB encoding, including where ζ places each logical qubit (Section IV-E), rather than from compensation learned by the optimizer. At each seed, the trained run classifies all 4096 test states in a single noisy pass. Reported accuracies are means over these 10 runs. For the standard QCNN, the error bars reflect variation across both trainings and noise realizations. For the encoded QCNN, the 10 runs implement the same classifier up to an overall scale, so its results describe a single trained model, and we attribute its error bars to variation over noise realizations.

B. NOISELESS TRAINING OF STANDARD QCNN AND ENCODED QCNN

For the standard QCNN, the loss rises sharply within the first 10 iterations in several runs, reaching up to approximately 1.4, which is consistent with the learning rate of the bold-driver rule being initialized at $\eta = 1$ before it is halved after each increase in loss. All 10 runs then settle by approximately iteration 15 and converge to a final loss of approximately 0.096. The encoded QCNN shows smaller initial fluctuations, with the loss staying below approximately 0.75, and plateaus by approximately iteration 30 at a final loss of approximately 0.29. Its 10 curves are nearly indistinguishable after the first iterations, consistent with all initializations converging to the same classifier up to an overall scale (Section II-A2). Fig. 3 shows the noiseless training loss of the 10 runs of each model.

The final training loss of the encoded QCNN is approximately three times that of the standard QCNN, although both reach the same noiseless test accuracy at their fitted thresh-

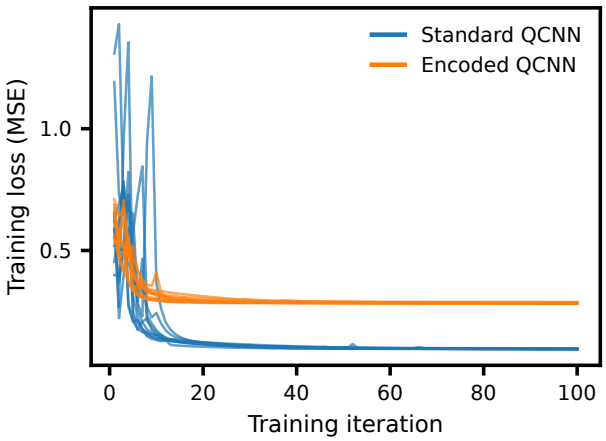


FIGURE 3: Training loss (MSE of (5)) over 100 iterations of noiseless statevector training for the standard QCNN and the encoded QCNN. Each curve is one of the 10 independent random initializations of Section II-A2.

olds (Section IV-C). This is consistent with the compressed output range of the encoded QCNN, whose maximum test output of approximately 0.70 lies well below the SPT label of 1, so the MSE of (5) cannot approach zero regardless of how well the outputs separate the two phases. The training loss therefore measures how closely the outputs match the labels $\{0, 1\}$, not how well the two classes are separated, and the fitted threshold recovers the separation that the loss does not reflect.

C. NOISELESS CLASSIFICATION AND ERROR PROFILES

All results in this section are obtained at inference only. At $p = 0$ and at the fitted decision thresholds of Section II-A2, the two models reach similar test accuracy. The encoded QCNN reaches 0.8132, identical across all 10 runs, and the standard QCNN reaches 0.8154 ± 0.0030. Encoding therefore does not cost noiseless accuracy, and the accuracy gap between the two architectures opens only under noise. Comparable accuracy, however, conceals different error profiles, shown in Table 1. Of the 4096 test states, 1677 ($\approx 41\%$) lie in the SPT phase. The encoded QCNN labels 23.6% of states as SPT, with specificity 0.988 and precision 0.971 but recall 0.561. The standard QCNN labels 55.9% of states as SPT, with recall 0.958 but specificity 0.717 and precision 0.701. The two classifiers thus reach comparable accuracy by erring in opposite directions. The encoded QCNN misses SPT states, while the standard QCNN assigns the SPT label to states outside the phase.

Both models are evaluated at their fitted thresholds. For reference, at the default threshold of 0.5, the encoded QCNN reaches a noiseless test accuracy of 0.703 and a training accuracy of 0.375, against 0.812 and 0.928 for the standard QCNN. Averaged over the 10 runs, the minimum and maximum test outputs of the encoded QCNN are approximately -0.16 and 0.70, with little variation across runs, compared with -0.26 and 1.00 for the standard QCNN, whose per-run minimum ranges from -0.73 to approximately 0. The encoded outputs are compressed toward the lower part of the range. On the training set its largest output is 0.498, so at a threshold of 0.5 it labels none of the 25 SPT training states as SPT, while at its fitted threshold its training accuracy is 0.85. On the test set it labels only 11.2% of states as SPT at a threshold of 0.5, with recall 0.274. The error directions do not arise from the fitted thresholds as they hold at the default threshold as well, with recall 0.274 and specificity 1.000 for the encoded QCNN against recall 0.897 and specificity 0.753 for the standard QCNN. All other accuracies reported in this paper therefore use the fitted thresholds.

TABLE 1: Noiseless ($p = 0$) confusion matrices on the 4096-state test set (1677 SPT, 2419 non-SPT), with SPT as the positive class. Counts and rates are means over 10 runs. Thr. is the decision threshold, where fitted thresholds are the values of Section II-A2. Rec., Spec., and Acc. denote recall, specificity, and accuracy.

Model	Thr.	TP	FN	FP	TN	Rec.	Spec.	Acc.
Encoded	fitted	940.0	737.0	28.0	2391.0	0.561	0.988	0.813
Standard	fitted	1605.9	71.1	685.1	1733.9	0.958	0.717	0.815
Encoded	0.5	459.5	1217.5	0.0	2419.0	0.274	1.000	0.703
Standard	0.5	1504.8	172.2	598.0	1821.0	0.897	0.753	0.812

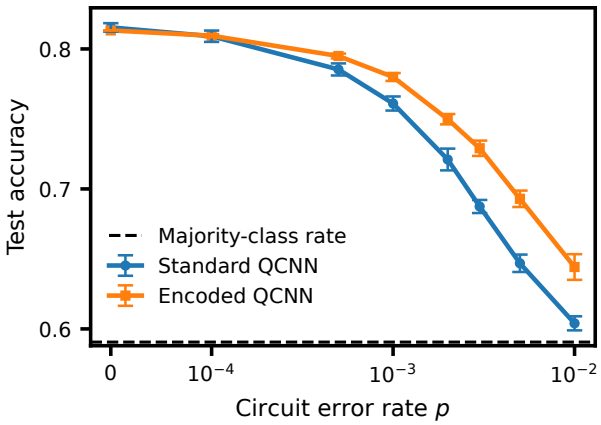


FIGURE 4: Test accuracy of the standard QCNN and the encoded QCNN on the 4096-state test set as a function of the circuit error rate p, at the fitted decision thresholds of Section II-A2. Weights and thresholds are fixed from noiseless training. Points are means over 10 runs, and error bars show one standard deviation.

D. CLASSIFICATION UNDER CIRCUIT-LEVEL NOISE

Both accuracies decrease monotonically with noise p. The encoded QCNN is more accurate than the standard QCNN at every error rate $p \geq 0.0005$, and the two are indistinguishable

at $p = 10^{-4}$. At that lowest rate, the standard QCNN loses 0.0063 in accuracy relative to $p = 0$, while the encoded QCNN loses 0.0038. The accuracy gap between the two models widens from 0.0004 at $p = 10^{-4}$ to 0.019 at $p = 10^{-3}$ and reaches its maximum of 0.046 at $p = 0.005$, where the encoded QCNN attains 0.693 ± 0.006 against 0.647 ± 0.006 for the standard QCNN. At $p = 0.01$ the gap narrows to 0.040 (0.644 ± 0.009 against 0.604 ± 0.005). Fig. 4 shows the test accuracy of both models as p is swept.

The encoded QCNN at error rate p matches or exceeds the standard QCNN at an error rate between approximately $p/1.9$ and $p/1.5$. At $p = 0.003$, the encoded QCNN reaches 0.729 ± 0.006, against 0.721 ± 0.008 for the standard QCNN at $p = 0.002$, and at $p = 0.005$ the encoded QCNN reaches 0.693 ± 0.006, against 0.688 ± 0.005 for the standard QCNN at $p = 0.003$. Linearly interpolating the standard QCNN curve between the swept error rates, the error rate at which it matches the accuracy of the encoded QCNN is smaller than that of the encoded QCNN by a factor between 1.5 and 1.9 for $p \in [0.0005, 0.01]$, with the factor generally increasing with p.

At $p = 0.01$, the standard QCNN accuracy of 0.604 ± 0.005 lies 0.013 above $2419/4096 \approx 0.591$, which is the accuracy of a classifier that labels every test state as non-SPT. At this error rate, the standard QCNN provides little information about the phase beyond the class proportions of the test set, whereas the encoded QCNN remains 0.054 above that level.

For the standard QCNN, the standard deviation ranges from 0.0041 to 0.0078 and includes variation across trainings. For the encoded QCNN, the standard deviation grows from 0.0011 at $p = 10^{-4}$ to 0.0092 at $p = 0.01$, which we attribute to noise realizations. The accuracy gap exceeds the standard deviation of either model by a factor of 2.2 at $p = 0.0005$, by more than 3.5 at $p = 0.001$ and $p = 0.002$, and by more than 4 at every $p \geq 0.003$.

For reference, at the default threshold of 0.5, the encoded QCNN is less accurate than the standard QCNN for $p \leq 0.005$ and more accurate at $p = 0.01$, reaching 0.615 against 0.606. The test MSE of (5), which does not depend on a threshold but measures agreement with the labels rather than class separation (Section IV-B), shows a similar crossover, where the encoded QCNN has the higher MSE for $p \leq 0.002$ (0.262 against 0.224 at $p = 0.001$), consistent with its compressed output range at $p = 0$, and the lower MSE for $p \geq 0.003$ (0.386 against 0.402 at $p = 0.005$ and 0.426 against 0.456 at $p = 0.01$).

The recall of the standard QCNN falls from 0.958 at $p = 0$ to 0.740 at $p = 0.001$ and 0.109 at $p = 0.01$, while its specificity rises from 0.717 to 0.947. The recall of the encoded QCNN falls from 0.561 to 0.476 at $p = 0.001$ and 0.134 at $p = 0.01$, while its specificity, already 0.988 at $p = 0$, rises to 0.998. Interpolating linearly between the swept error rates, the recall of the encoded QCNN falls to half its noiseless value at $p \approx 0.0045$, compared with $p \approx 0.0028$ for the standard QCNN. The standard QCNN has the higher recall for $p \leq 0.005$, a consequence of its higher noiseless recall, and the encoded QCNN has the higher recall only at $p = 0.01$ (0.134 ± 0.022 against 0.109 ± 0.007).

Precision behaves differently in the two models. The precision of the encoded QCNN stays between 0.970 and 0.975 at every error rate tested, so the SPT labels it does assign remain reliable as p increases. The precision of the standard QCNN falls from 0.701 at $p = 0$ to 0.661 at $p = 0.005$ and 0.589 at $p = 0.01$.

Because the two models operate at different points of the recall-specificity trade-off, we also evaluate, as a diagnostic, the standard QCNN at a threshold matched to the noiseless specificity of the encoded QCNN. For each standard QCNN run, the threshold is set so that its specificity on the noiseless test set equals 0.988 (mean threshold 0.794 ± 0.016), and is then held fixed as p is swept. At $p = 0$, this matched standard QCNN reaches an accuracy of 0.797 ± 0.007 and a recall of 0.520 ± 0.016, below the 0.813 and 0.561 of the encoded QCNN. Under noise, the encoded QCNN remains more accurate at every error rate, with the gap growing from 0.019 at $p = 10^{-4}$ to 0.049 at $p = 0.005$, and the recall of the matched standard QCNN falls to half its noiseless value at $p \approx 0.0026$, compared with $p \approx 0.0045$ for the encoded QCNN, and to 0.040 ± 0.003 at $p = 0.01$. Measured from each model's own noiseless accuracy, the matched standard QCNN loses the same accuracy as the encoded QCNN at an error rate smaller by a factor between 1.4 and 1.5 at every error rate tested. Because the matched threshold is set using the noiseless test labels, this comparison serves as a diagnostic of the operating point rather than as a reported accuracy. At equal noiseless specificity, the encoded QCNN therefore retains its advantage in both accuracy and recall, so the advantage of Fig. 4 does not arise from the choice of operating point.

E. COMPARISON WITH NON-BB ENCODINGS

To test whether the noise resilience of Section IV-D depends on the structure of the BB code or only on distributing each logical qubit over several physical qubits, we evaluate two control encodings under the same protocol, a random control and a repetition control.

The random control keeps the size of ζ and discards its structure. It uses the same 11-qubit register as $\mathcal{D}$, and each logical qubit q receives a support of the same weight $|\zeta(q)|$ it has under ζ, so both encodings fan out every logical gate onto the same number of physical qubits. Each support is drawn uniformly at random and independently of the others, so supports may overlap as they do under ζ. A draw is accepted only if the supports together cover all 11 qubits and are linearly independent over $\mathbb{F}_2$, which keeps the encoding injective. The supports are drawn once with a fixed seed of 0 and reused in every run, giving $P_0 = \{0, 2, 3, 4, 5, 7\}$, $P_1 = \{3, 4, 5, 6, 7, 8\}$, $P_2 = \{0, 3, 5, 9\}$, and $P_3 = \{0, 1, 4, 5, 8, 10\}$, with weights $(6, 6, 4, 6)$ matching those of ζ in (8). The logical readout is a linear map over $\mathbb{F}_2$ that inverts the encoding, and for each logical bit we use the

TABLE 2: Test accuracy on the 4096-state test set at the fitted decision thresholds, as a function of the circuit error rate p. Entries are means over 10 runs with $\pm$ one standard deviation. The majority-class accuracy is $2419/4096 \approx 0.591$.

p	Standard	BB Encoded	Random	Repetition
0	0.815 ± 0.003	0.813 ± 0.000	0.832 ± 0.025	0.793 ± 0.008
0.0001	0.809 ± 0.004	0.809 ± 0.001	0.800 ± 0.022	0.778 ± 0.007
0.0005	0.785 ± 0.004	0.795 ± 0.002	0.705 ± 0.013	0.728 ± 0.007
0.001	0.761 ± 0.005	0.780 ± 0.003	0.645 ± 0.006	0.682 ± 0.007
0.002	0.721 ± 0.008	0.750 ± 0.004	0.603 ± 0.002	0.631 ± 0.005
0.003	0.688 ± 0.005	0.729 ± 0.006	0.593 ± 0.001	0.610 ± 0.003
0.005	0.647 ± 0.006	0.693 ± 0.006	0.591 ± 0.000	0.595 ± 0.002
0.01	0.604 ± 0.005	0.644 ± 0.009	0.591 ± 0.000	0.591 ± 0.000

minimum-weight such map, which gives the control the most favorable linear readout available to it.

The repetition control assigns each logical qubit three dedicated physical qubits, giving four disjoint supports $\{3q, 3q+1, 3q+2\}$ for $q = 0, \ldots, 3$ on a 12-qubit register. The supports are disjoint so every logical CNOT becomes three physical CNOTs between distinct qubits. Each logical bit is read out by majority vote over its three physical qubits.

For both controls, the encoded input and the fan-out constructions of (9) and (10) are built from the control mapping in the same way as for ζ, and only the readout differs as described above. Each control is trained under noiseless statevector simulation for 100 iterations from 10 independent random initializations, and its weights and fitted decision thresholds are held fixed as p is swept under the noise model given in Section IV-A. Weights are not shared between encodings, since each mapping yields a different encoded circuit whose parameters must be learned on that circuit (Section III-E).

Under the counting of Section IV-A, the random control has 530 single-qubit gates and 240 CNOTs at depth 196, and 1513 of its 1916 fault locations can reach its output (435 single-qubit, 214 CNOT, and 864 idle). The repetition control has 291 single-qubit gates and 174 CNOTs at depth 93, three copies of each gate in the standard QCNN, and 855 of its 942 fault locations can reach its output (288 single-qubit, 174 CNOT, and 393 idle).

Table 2 lists the test accuracy of the standard QCNN, the encoded QCNN, the random control, and the repetition control at each error rate. At $p = 0$, both controls are able to classify the phase. Reporting each value as the mean $\pm$ one standard deviation over the 10 runs, the random control reaches 0.832 ± 0.025 and the repetition control 0.793 ± 0.008, against 0.813 for the BB encoded QCNN. Unlike the BB encoded QCNN, the 10 runs of each control do not converge to a single classifier, so their error bars include variation across trainings. This suggests encoding the logical qubits over several physical qubits does not alone cost noiseless accuracy.

However, under noise, both the random and repetition controls lose accuracy faster than the BB encoded QCNN and the standard QCNN. The random control falls to 0.705 ± 0.013 at $p = 0.0005$ and 0.645 ± 0.006 at $p = 0.001$, and lies within 0.003 of the majority-class accuracy of 0.591 for $p \geq 0.003$. The repetition control falls to 0.728 ± 0.007 at $p = 0.0005$ and 0.682 ± 0.007 at $p = 0.001$, and lies within 0.005 of 0.591 for $p \geq 0.005$. At $p = 0.01$, both controls are at 0.591, while the BB encoded QCNN remains 0.054 above that level. Although the random control starts 0.019 above the BB encoded QCNN at $p = 0$, it falls below it by $p = 10^{-4}$. The BB encoded QCNN is more accurate than both controls at every $p \geq 10^{-4}$, with the largest gaps of 0.147 over the random control at $p = 0.002$ and 0.119 over the repetition control at both $p = 0.002$ and $p = 0.003$.

Applying the linear interpolation of Section IV-D, the error rate at which the repetition control matches the accuracy of the BB encoded QCNN is smaller than that of the BB encoded QCNN by a factor between 5.7 and 11.2 for $p \in [0.001, 0.01]$, largest at $p = 0.001$ and between 5.7 and 6.1 for $p \geq 0.002$, and the corresponding factor for the random control is between 4.2 and 9.7 for $p \in [0.0005, 0.01]$, growing with p. Because the four models start from different noiseless accuracies, we also measure the fraction of each model's noiseless accuracy above 0.591 that it retains under noise. At $p = 0.001$, the BB encoded QCNN retains 0.85 of this margin, the standard QCNN 0.76, the repetition control 0.45, and the random control 0.23. Interpolating linearly, each model retains half of this margin at $p \approx 0.0045$ for the BB encoded QCNN, $p \approx 0.0025$ for the standard QCNN, $p \approx 0.00089$ for the repetition control, and $p \approx 0.00047$ for the random control. Both controls are therefore less noise resilient than the unencoded standard QCNN, by factors of approximately 2.9 for the repetition control and 5.4 for the random control at half the retained margin, and less noise resilient than the BB encoded QCNN by factors of approximately 5.1 and 9.6, respectively.

Within the encodings tested, the noise resilience of the BB encoded QCNN does not seem to be explained by encoding into more physical qubits, since both controls are less noise resilient than the standard QCNN. Two measured properties follow the noise resilience of the encoded models. The first is the number of output-reachable fault locations, which orders all four models in the same way as the error rate at which they retain half their margin, from the BB encoded QCNN (213) through the standard QCNN (285) and the repetition control (855) to the random control (1513). The second is the output margin. At $p = 0$, the mean difference between the maximum test output and the decision threshold is 0.31 for the BB

encoded QCNN, 0.15 for the repetition control, and 0.04 for the random control, the same order as their noise resilience. Margin alone does not set the ordering across all four models, however, since the standard QCNN has the largest margin, 0.56, and is less noise resilient than the BB encoded QCNN. The reachable count is not an alternative to the BB structure but one way that structure acts on the output. Under ζ, the index-wise pairing of (10) leaves only data qubits 7 and 10 exchanging gates with readout qubit 8, so few fault locations can propagate to the readout. The random control has the same support weights on the same 11-qubit register, yet 1513 of its locations can reach its output, so the reduction comes from where ζ places each logical qubit and not from the size of its supports. Whether the reachable count fully accounts for the noise resilience, or whether the output margin or other properties of the encoded circuit also contribute, is left to future work, for example by injecting faults only on output-reachable locations or by comparing the models at matched margins.

F. QUBIT RESOURCE ANALYSIS

The toric code encodes logical qubits into physical qubits scaling as $n \propto d^2$. Reported circuit-level thresholds range from $\sim 0.5\%$ to $\sim 1.1\%$ per gate, depending on the noise model, syndrome circuit, and decoder [28], [29]. The total qubit count scaling is given as $n = 2d^2$ per code block encoding $k = 2$ logical qubits, where d is the code distance and increasing d exponentially suppresses logical errors. With minimum distance d, the code can correct arbitrary errors on up to $\lfloor \frac{d-1}{2} \rfloor$ qubits. Encoding $k = 4$ logical qubits at distance $d = 4$, the distance of the $[[18, 4, 4]]$ code, therefore requires two blocks of $2d^2 = 32$ data qubits, or 64 data qubits for memory alone (128 including one check ancilla per stabilizer). The equivalent count using four rotated surface-code patches is $4(2d^2 - 1) = 124$.

In contrast, the $[[18, 4, 4]]$ BB code encodes the same 4 logical qubits into 18 data qubits, or 36 physical qubits including one check ancilla per stabilizer, a $3.6\times$ reduction in memory footprint relative to the toric code at either count and $3.4\times$ relative to four rotated surface-code patches. The 29-qubit register of the error-corrected QCNN (Appendix B) is smaller because it omits the seven data qubits outside the supports of ζ (Section III-A). These qubits remain in the supports of the code's checks, so a complete $[[18, 4, 4]]$ memory retains all 36. Comparable reductions have been reported elsewhere in the bicycle family, where four logical qubits at a distance 5 occupy 60 physical qubits against 200 for a surface code at the same parameters [30]. Bravyi et al. report a circuit-level threshold of 0.7% for the larger codes of the BB family [13].

The Eastin-Knill theorem rules out a universal transversal gate set for any quantum error-correcting code that detects arbitrary single-qubit errors [22], so the arbitrary-angle rotations of the QCNN's variational layers generally require a non-transversal implementation in either code. In the surface code, they are supplied by magic state distillation [31], where a single 15-to-1 factory requires roughly 800 to 1,150 physical qubits at a physical error rate of 10^{-4} and several thousand at 10^{-3} [23], which is closer to current gate error rates [32]. Full fault-tolerant algorithms are estimated to require between 10^4 and 10^6 physical qubits in total [33]–[36]. Distillation factories have since been proposed inside a single BB code block [15], supplying the non-Clifford resource without leaving the qLDPC architecture, which we do not adopt here. The encoded QCNN does not incur this overhead because it does not implement fault-tolerant non-Clifford logic. Its layers are fanned out gate by gate over the physical supports of ζ (Section III-C) and are not logical operations of the code (Section III-E), so the comparison above is between encoded memories, not between fault-tolerant implementations of the QCNN. The interleaved coherent-feedback decoder of Appendix B is intended to absorb the residual correction burden left by non-transversal rotations, and how it performs is left to future work.

V. CONCLUSION AND FUTURE DIRECTIONS

In this study, we considered the degradation of a noiselessly trained QCNN's classification accuracy under NISQ circuit-level noise. We proposed the integration of the BB code to address this issue. The $[[18, 4, 4]]$ BB code provides a mapping for physical to logical qubits via sparse parity check matrices to encode a 4-qubit QCNN into an 11-qubit encoded QCNN. With both models trained noiselessly and their weights and decision thresholds held fixed as the circuit error rate is swept, the encoded QCNN reaches the same SPT phase classification accuracy as the unencoded QCNN at 1.5 to 1.9 times the error rate, attaining an accuracy of 0.693 at $p = 0.005$, against 0.688 for the unencoded QCNN at $p = 0.003$. This noise resilience is obtained from the encoding alone, without syndrome extraction or a decoder, such as BP-OSD. No training is performed under noise, so neither model can adapt to it. Control encodings that also spread each logical qubit over several physical qubits lose accuracy faster than both the encoded QCNN and the standard QCNN, so within the encodings tested the noise resilience is not explained by redundancy alone. A natural next step is the error-corrected QCNN of Appendix B, whose coherent-feedback decoder would add active correction to the encoded circuit. By co-training the decoder alongside the QCNN variational parameters or by initializing from a pre-trained QCNN model, the correction layer can be exposed to the same error distribution encountered during inference, aligning the noise seen at training time with that seen in application. Such co-training also requires preparing the input in the code space as the encoded input satisfies the Z-type but not the X-type checks (Section III-E). The X-type syndromes are not trivial even in the absence of noise, and only a code-space input ensures that the syndromes seen by the decoder reflect faults alone.

One open question raised in Section III-A bears directly on that program. The encoded two-qubit layers are not logical operations so the intra-block logical CNOT on the

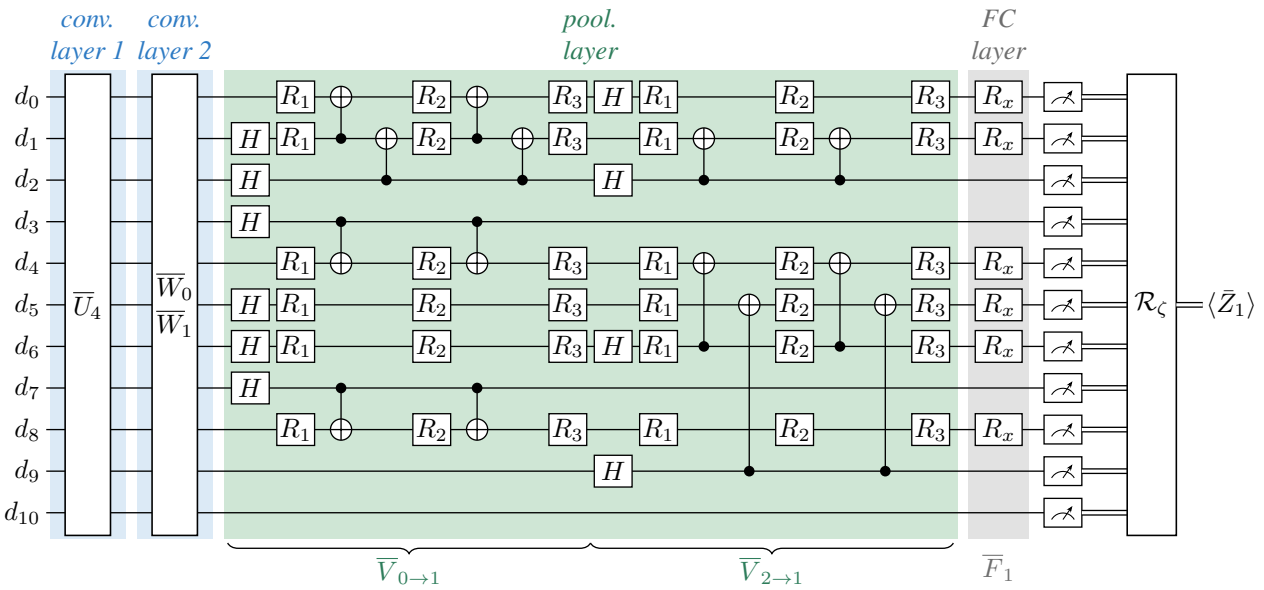


FIGURE 5: Gate-level circuit of the encoded QCNN $\overline{\mathcal{C}}_{\text{QCNN}}$ on the 11 qubits $d_0, \ldots, d_{10}$ of $\mathcal{D}$. The convolution layers are shown as the blocks $\overline{U}_4$ and $\overline{W}_0\overline{W}_1$, the latter applying $\overline{W}_0$ and then $\overline{W}_1$. In the pooling layers $\overline{V}_{0\to1}$ and $\overline{V}_{2\to1}$, each single-qubit gate is fanned out over the support of its logical qubit by (9), and each CNOT is applied pairwise between the supports of its control and target by (10), omitting pairs with $j = k$. Each controlled rotation V_i, with trainable parameters $(\theta_i, \phi_i, \lambda_i)$, is applied to its target as R_1, CNOT, R_2, CNOT, R_3, where $R_1 = R_z\big((\lambda_i - \phi_i)/2\big)$, $R_2 = R_y(-\theta_i/2)\, R_z\big(-(\phi_i + \lambda_i)/2\big)$, and $R_3 = R_z(\phi_i)\, R_y(\theta_i/2)$, so that the target undergoes $R_z(\phi_i)\, R_y(\theta_i)\, R_z(\lambda_i)$ when the control is $|1\rangle$ and the identity otherwise. The fully connected layer $\overline{F}_1$ applies R_x to the support $\mathcal{P}_1$ of logical qubit 1. The logical readout $\mathcal{R}_\zeta$ of Section III-D is then applied to the final state of $\mathcal{D}$ and returns $\langle \bar{Z}_1 \rangle$. No check qubits or syndrome extraction are included. The same fan-out operation is applied to the convolutional layers.

$[[18, 4, 4]]$ code would require a non-transversal construction, and accounting for its depth or ancilla cost would revise the resource comparison of Section IV-F. The encoded QCNN results reported here were obtained under the encoding as implemented, and we regard supplying a valid intra-block logical CNOT as a prerequisite to scaling this architecture to QEC.

A further limitation is that the encoded input states are prepared noiselessly (Section III-B). On hardware, state preparation is itself noisy, since preparing many-body ground states is expected to require substantially deeper circuits than the QCNN that processes them [6], so the noise resilience reported here does not account for preparation errors.

.

APPENDIX A GATE-LEVEL CIRCUIT OF THE ENCODED QCNN

Fig. 5 gives the gate-level circuit of the encoded QCNN $\overline{\mathcal{C}}_{\text{QCNN}}$ on the 11 qubits of $\mathcal{D}$, complementing the gate supports shown in Fig. 2(b).

APPENDIX B ERROR-CORRECTED QCNN EXTENSION

A. CHECK OPERATORS AND SYNDROME EXTRACTION

1) Check Operators

The bivariate polynomials are a pair of matrices

$$A = A_1 + A_2 + A_3 \quad \text{and} \quad B = B_1 + B_2 + B_3, \quad (17)$$

such that each matrix A_i and B_j is a power of x or y, while $\mathbf{H}^X$ and $\mathbf{H}^Z$ each have size $(n/2) \times n$. Each row $v \in \mathbb{F}_2^n$ of $\mathbf{H}^X$ defines an X-type operator, and each row $v \in \mathbb{F}_2^n$ of $\mathbf{H}^Z$ defines a Z-type operator, given by

$$X(v) = \prod_{j=1}^{n} X_j^{v_j} \quad \text{and} \quad Z(v) = \prod_{j=1}^{n} Z_j^{v_j}, \quad (18)$$

respectively.

2) Syndrome Extraction

The standard syndrome measurement (SM) circuit of $\text{QC}(A, B)$ repeatedly measures the syndrome of each check operator, which requires n data qubits and n ancillary check qubits to record the measured syndromes, for a total of $2n$ physical qubits. The SM circuit only applies CNOTs to pairs of qubits that are connected in the Tanner graph [37].

The SM circuit $\mathcal{U}$ starts and ends with a code-dependent initialization and measurement cycle that determines the logical qubit initialization suited for the initial state and measures logical qubits in a proper basis. The remainder of the SM circuit consists of N_c syndrome cycles (SC), which are periodically repeated throughout the circuit. Each SC is responsible for measuring syndromes for all n check operators of the code.

B. COHERENT-FEEDBACK DECODER

The encoded QCNN of Section III-C carries no syndrome extraction or correction. As an extension, we describe an error-corrected QCNN that interleaves a coherent-feedback decoder between the encoded layers. We do not evaluate this extension here and leave its simulation to future work.

The proposed decoder is interleaved between each convolutional, pooling, and fully connected layer of the QCNN and the next. Syndrome-based neural decoders [38], [39] consume a measured syndrome bit string and return a correction that is subsequently applied by classical control. The decoder proposed here keeps the syndrome in the quantum register throughout and never reads it out. Thus, rather than correcting the non-Clifford rotations themselves, the decoder is realized as a coherent-feedback layer, which is a parameterized quantum sub-circuit inserted inside each syndrome cycle, between the extraction ladder and the ancilla reset. Once the check ancillas have accumulated the parities of their Tanner neighborhoods, each ancilla applies a trainable $R_x(\gamma_i)$ rotation and then fans out CX (X-checks) or CZ (Z-checks) onto the data qubits in its support, so that the correction is applied coherently before the next layer executes. The ancilla reset that follows is the only non-unitary operation in the cycle and no syndrome is read out at any point. This design is intended to confine error propagation to individual layers rather than allowing it to compound across the full circuit depth.

In the error-corrected QCNN, the 18 check ancillas are initialized in the product state

$$|\Psi_0\rangle \;=\; |0\rangle_X^{\otimes 9} \otimes |\overline{\psi}\rangle_{\mathcal{D}} \otimes |0\rangle_Z^{\otimes 9}, \tag{19}$$

which the decoder resets to after each syndrome extraction, so that (19) is also the ancilla state entering every subsequent layer.

The evolved state occupies a Hilbert space of dimension $2^{n_{\text{total}}}$, with $n_{\text{total}} = \ell m + |\mathcal{D}| + \ell m = 9 + 11 + 9 = 29$ for the $\ell m = 9$ X-checks, $|\mathcal{D}| = 11$ data qubits, and $\ell m = 9$ Z-checks. The data-qubit marginal is obtained by summing the computational-basis probabilities of the final state over all ancilla outcomes, and the logical readout of Section III-D is applied to that marginal.

The decoder's $2\ell m = n = 18$ rotation angles are trained with SPSA on logical-state preservation over all 16 logical basis states, as detailed in Appendix B-C.

C. TRAINING THE COHERENT-FEEDBACK DECODER

This training procedure is implemented in our simulation code, but its results are not evaluated in this work. In the error-corrected QCNN of Appendix B, the coherent-feedback correction occurs between each hidden layer and the next variational layer so the parameter updates computed by the finite-difference scheme reflect gradient estimates over a cleaner logical state. If correction were deferred to the end of the circuit, then the noise would accumulate over the circuit, creating harder to detect and correct errors. In practice, this benefit depends on the decoder distinguishing Pauli errors from the intended operations of the QCNN. The procedure below trains the decoder on Clifford circuits under Pauli noise rather than on the variational rotations that occur in the QCNN, so a decoder trained this way may treat those rotations as errors and correct against them, which motivates the joint training discussed in Section V.

The coherent-feedback process avoids the classical decoding step of BP-OSD, which in real-time operation must keep pace with syndrome cycles of order $1\,\mu$s on superconducting processors [40], [41]. Current decoders trade accuracy for latency to meet this decoding budget [42]. When the decoder falls behind, physical qubits must sit idle between layers, during which T_1 (energy relaxation) and T_2 (dephasing) processes accumulate additional errors on the quantum hardware.

The learnable parameters of the coherent-feedback layer are attached to the check nodes of the Tanner graph associated with the BB code, where each of the ℓm X-checks and ℓm Z-checks carries a single rotation angle, shared across all edges incident on that check, giving $2\ell m = n = 18$ trainable parameters for the $[[18, 4, 4]]$ code. Because the fan-out follows the rows of H_X (with $H_Z = H_X$ for this code), the connectivity of the check and variable nodes adheres to the code's locality, thereby reflecting the geometric constraints imposed on the encoded gates. The check ancillas are reset after the correction rather than before the following extraction ladder. Therefore, parities accumulate across two ladders before the next correction fires. The decoder therefore acts on the difference between consecutive syndromes rather than on the absolute syndrome history, in the manner of a detector. Its input is therefore not a classical bit string but the joint ancilla, given the data state left by syndrome extraction, and its output is not a predicted binary correction vector but a coherent, amplitude-modulated application of each check's own support to the data block. Because the syndrome is never collapsed, the decoder realizes a fixed map from ancilla state to correction whose strength on check i is set by the learned angle γ_i.

Specifically, the four logical qubits admit $2^4 = 16$ distinct computational basis states, and the training set is

$$\mathcal{F}_{\text{train}} \;=\; \big\{\, y \in \{0,1\}^4 \big\}, \tag{20}$$

where each element $y = (y_1, y_2, y_3, y_4)$ labels one logical basis state. At each SPSA iteration every $y \in \mathcal{F}_{\text{train}}$ is prepared by applying X gates to the corresponding data qubits and encoded through ζ into $|\overline{y}\rangle \in (\mathbb{C}^2)^{\otimes 11}$ via fan-out gate operations. The encoded state is propagated through the full syndrome-extraction and coherent-feedback correction circuit under stochastic Pauli noise. The 11 data qubits are then measured into a classical register and projected through L_X to yield a predicted logical label $\hat{y}$. The task is thus logical-state preservation, with the objective of recovering each prepared basis state unchanged.

Given that the logical state space is finite and small, the loss in (21) is computed over all $|\mathcal{F}_{\text{train}}| = 16$ patterns at every iteration, making each update equivalent to a full-batch

gradient step across the complete input distribution. This exhaustive evaluation assesses the trained parameters $\boldsymbol{\gamma}$ on every logical basis state, and a loss of 0 indicates that all 16 basis states were recovered in that evaluation. Because both the training states and the readout are in the Z basis, this objective may give the Z-check angles little training signal.

To correct errors, we augment the syndrome extraction circuit with the parametrized coherent-feedback layer introduced in Appendix B-B. This layer introduces $2\ell m = n = 18$ trainable rotation angles $\gamma_i \in [0, 2\pi)$, where $R_x(\gamma_i^X)$ and $R_x(\gamma_i^Z)$ are applied to the ith X-check and Z-check ancilla qubits, respectively, once syndrome extraction has accumulated the parity of each check's Tanner neighborhood. Each rotated ancilla then acts as the control of a fan-out of CX (for X-checks) or CZ (for Z-checks) gates onto the data qubits in its support, so that γ_i modulates the amplitude with which check i applies its correction. Because the fan-out is restricted to the 11 physical indices of the collapsed data register, each check applies a truncation of its weight-6 row.

The objective used during training is the logical state or logical label. The set of true logical labels $\{y_i\}_{i=1}^{|\mathcal{F}_{\text{train}}|}$ and the corresponding predicted logical labels $\{\hat{y}_i\}_{i=1}^{|\mathcal{F}_{\text{train}}|}$ given in $\mathcal{F}_{\text{train}}$ returned by the decoder after circuit execution, the loss is given as

$$\texttt{Loss}(\boldsymbol{\gamma}) \;=\; \frac{1}{|\mathcal{F}_{\text{train}}|} \sum_{i=1}^{|\mathcal{F}_{\text{train}}|} \mathbf{1}[\hat{y}_i \neq y_i]\,, \tag{21}$$

where $\boldsymbol{\gamma}$ denotes the trainable parameters of the coherent-feedback layer, which are distinct from the QCNN circuit parameters $\boldsymbol{\theta}$, and $\mathbf{1}[\cdot]$ denotes the indicator function. The loss lies in $[0, 1]$, where 0 indicates perfect recovery and 1 indicates that every logical state is mislabeled. This normalization provides the SPSA gradient estimator with a stationary and bounded signal that is independent of the number of test cases and code distance.

The correction layer is embedded inside a quantum circuit evaluated by a noisy stochastic simulator [43]. Because the loss function is a non-differentiable indicator function, the gradients of $\texttt{Loss}$ with respect to the trainable parameters $\boldsymbol{\gamma}$ cannot be computed analytically [44]. Therefore, SPSA is employed as the optimizer, such that at each iteration τ, the algorithm evaluates the loss 3 times, each over all 16 basis states (48 circuit executions per iteration), regardless of the number of parameters $2\ell m$, making it practical for circuits with many syndrome checks.

Each training iteration proceeds in four stages.

1) **Center evaluation.** The unperturbed circuit is evaluated to obtain the center-point loss $\texttt{Loss}_0 = \texttt{Loss}(\boldsymbol{\gamma}^{(\tau)})$, where $\boldsymbol{\gamma}^{(\tau)}$ is the weight vector at the τth iteration and is recorded for the learning-rate adaptation rule described below.
2) **Positive perturbation.** A simultaneous perturbation vector $\boldsymbol{\Delta} \in \{-1, +1\}^{2\ell m}$ is drawn independently and uniformly at random. The parameters are shifted to $\boldsymbol{\gamma}^{(\tau)} + c\,\boldsymbol{\Delta}$, where $c = 0.2$ rad., is the perturbation magnitude, and the circuit is evaluated to obtain $\texttt{Loss}_+$.
3) **Negative perturbation.** The parameters are shifted to $\boldsymbol{\gamma}^{(\tau)} - c\,\boldsymbol{\Delta}$. Using the same realization of $\boldsymbol{\Delta}$, the circuit is evaluated to obtain $\texttt{Loss}_-$.
4) **Gradient step.** The SPSA gradient estimate for parameter i is given by (22),

$$\hat{g}_i \;=\; \frac{\texttt{Loss}_+ - \texttt{Loss}_-}{2\,c\,\Delta_i}, \tag{22}$$

and the gradient descent step of (23) is applied,

$$\gamma_i^{(\tau+1)} \;=\; \left(\gamma_i^{(\tau)} - \alpha_\tau\,\hat{g}_i\right) \mod 2\pi, \tag{23}$$

where α_τ is the current learning rate. All parameters are updated simultaneously in a single step, with the base parameters restored from a snapshot taken before the perturbations were applied. Angles are wrapped modulo 2π throughout.

The learning rate α_τ is adapted at every step using the bold-driver rule [20]. After the gradient step, the center-point loss $\texttt{Loss}_0$ of the current iteration is compared with that of the previous one. If the loss decreased, the learning rate is scaled by 1.05, encouraging larger steps when progress is being made. If the loss has increased or remained constant, the learning rate is halved, contracting the search radius as,

$$\alpha_{\tau+1} \;=\; \begin{cases} 1.05\,\alpha_\tau & \text{if } \texttt{Loss}_0^{(\tau)} < \texttt{Loss}_0^{(\tau-1)}, \\ 0.5\,\alpha_\tau & \text{otherwise.} \end{cases} \tag{24}$$

All trainable parameters are initialized to zero so that each check ancilla undergoes an identity rotation, and the fan-out is applied at full amplitude, conditioned on the parity accumulated during syndrome extraction. Therefore, the first center-point evaluation measures the logical error rate under an unmodulated Tanner fan-out, which serves as the baseline against which the SPSA tunes per-check amplitudes. A seeded random number generator is used for the Bernoulli draws that produce the $\boldsymbol{\Delta}$ vectors, so that the training run is reproducible.

ACKNOWLEDGMENT

The authors would like to thank Dr. Jorma Kilpi and Dr. Olli Apilo of VTT Technical Research Centre of Finland for their insightful feedback to this work.

The authors acknowledge the use of AI-assisted tools, including Claude (Anthropic Claude Opus 5.5, October 2026 version), during the preparation of this manuscript. These tools were used for (i) assisting with LaTeX formatting of equations and tables, (ii) generating the TikZ source code for Figs. 2 and 5 from author-specified circuit layouts and qubit supports, and (iii) assisting with the implementation of a small number of functions in the author-written simulation code and adding function documentation. All AI-assisted code was reviewed and corrected by the authors. The authors take full responsibility for the content of this manuscript.

REFERENCES

[1] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," Nature, vol. 549, no. 7671, pp. 195–202, Sep. 2017, doi: 10.1038/nature23474.

[2] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," Phys. Rev. Lett., vol. 122, no. 4, p. 040504, Feb. 2019, doi: 10.1103/PhysRevLett.122.040504.

[3] J. M. Gambetta, J. M. Chow, and M. Steffen, "Building logical qubits in a superconducting quantum computing system," npj Quantum Inf., vol. 3, p. 2, Jan. 2017, doi: 10.1038/s41534-016-0004-0.

[4] I. Cong, S. Choi, and M. D. Lukin, "Quantum convolutional neural networks," Nat. Phys., vol. 15, no. 12, pp. 1273–1278, Dec. 2019, doi: 10.1038/s41567-019-0648-8.

[5] P. Bermejo, P. Braccia, M. S. Rudolph, Z. Holmes, L. Cincio, and M. Cerezo, "Quantum convolutional neural networks are effectively classically simulable," PRX Quantum, vol. 7, no. 2, p. 020304, Apr. 2026, doi: 10.1103/8qt9-72ts.

[6] P. Zapletal, N. A. McMahon, and M. J. Hartmann, "Error-tolerant quantum convolutional neural networks for symmetry-protected topological phases," Phys. Rev. Res., vol. 6, no. 3, p. 033111, Jul. 2024, doi: 10.1103/PhysRevResearch.6.033111.

[7] A. Y. Kitaev, "Fault-tolerant quantum computation by anyons," Ann. Phys., vol. 303, no. 1, pp. 2–30, Jan. 2003, doi: 10.1016/S0003-4916(02)00018-0.

[8] M. Gowrishankar, D. Claudino, J. Wright, and T. Humble, "Logical error rates for a [[4,2,2]]-encoded variational quantum eigensolver ansatz," 2024, arXiv:2405.03032.

[9] C. N. Self, M. Benedetti, and D. Amaro, "Protecting expressive circuits with a quantum error detection code," Nat. Phys., vol. 20, pp. 219–224, 2024, doi: 10.1038/s41567-023-02282-2.

[10] Z. He, D. Amaro, R. Shaydulin, and M. Pistoia, "Performance of quantum approximate optimization with quantum error detection," Commun. Phys., vol. 8, p. 217, May 2025, doi: 10.1038/s42005-025-02136-8.

[11] H. Goto, "Many-hypercube codes: High-rate quantum error-correcting codes for high-performance fault-tolerant quantum computing," Sci. Adv., vol. 10, p. eadp6388, 2024, doi: 10.1126/sciadv.adp6388.

[12] R. Nakai and H. Goto, "Subsystem many-hypercube codes: High-rate concatenated codes with low-weight syndrome measurements," 2025, arXiv:2510.04526.

[13] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, "High-threshold and low-overhead fault-tolerant quantum memory," Nature, vol. 627, pp. 778–782, Mar. 2024, doi: 10.1038/s41586-024-07107-7.

[14] T. J. Yoder, E. Schoute, P. Rall, E. Pritchett, J. M. Gambetta, A. W. Cross, M. Carroll, and M. E. Beverland, "Tour de gross: A modular quantum computer based on bivariate bicycle codes," 2025, arXiv:2506.03094.

[15] S. Xu, K. Liu, P. Rall, Z. He, and Y. Ding, "Distilling magic states in the bicycle architecture," in Proc. ACM/IEEE 53rd Annu. Int. Symp. Comput. Archit. (ISCA), Raleigh, NC, USA, Jun. 2026, pp. 2081–2095, arXiv:2602.20546.

[16] K. Wang, Z. Lu, C. Zhang et al., "Demonstration of low-overhead quantum error correction codes," Nat. Phys., vol. 22, pp. 308–314, Feb. 2026, doi: 10.1038/s41567-025-03157-4.

[17] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, May 2015, doi: 10.1038/nature14539.

[18] Y. LeCun and Y. Bengio, "Convolutional networks for images, speech, and time series," in The Handbook of Brain Theory and Neural Networks, M. A. Arbib, Ed. Cambridge, MA, USA: MIT Press, 1995, pp. 255–258.

[19] I. P. McCulloch, "Infinite size density matrix renormalization group, revisited," 2008, arXiv:0804.2509.

[20] R. Battiti, "First- and second-order methods for learning: Between steepest descent and Newton's method," Neural Comput., vol. 4, no. 2, pp. 141–166, Mar. 1992, doi: 10.1162/neco.1992.4.2.141.

[21] M. Hagiwara and H. Imai, "Quantum quasi-cyclic LDPC codes," in Proc. IEEE Int. Symp. Inf. Theory (ISIT), 2007, pp. 806–810, arXiv:quant-ph/0701020.

[22] B. Eastin and E. Knill, "Restrictions on transversal encoded quantum gate sets," Phys. Rev. Lett., vol. 102, no. 11, p. 110502, Mar. 2009, doi: 10.1103/PhysRevLett.102.110502.

[23] D. Litinski, "Magic state distillation: Not as costly as you think," Quantum, vol. 3, p. 205, Dec. 2019, doi: 10.22331/q-2019-12-02-205.

[24] N. P. Breuckmann and S. Burton, "Fold-transversal Clifford gates for quantum codes," Quantum, vol. 8, p. 1372, Jun. 2024, doi: 10.22331/q-2024-06-13-1372.

[25] J. N. Eberhardt and V. Steffan, "Logical operators and fold-transversal gates of bivariate bicycle codes," IEEE Trans. Inf. Theory, vol. 71, no. 2, pp. 1140–1152, Feb. 2025, doi: 10.1109/TIT.2024.3521638.

[26] H. Sayginel, S. Koutsioumpas, M. Webster, A. Rajput, and D. E. Browne, "Fault-tolerant logical Clifford gates from code automorphisms," PRX Quantum, vol. 6, no. 3, p. 030343, Sep. 2025, doi: 10.1103/vf7v-cpq9.

[27] A. Cowtan and S. Burton, "CSS code surgery as a universal construction," Quantum, vol. 8, p. 1344, May 2024, doi: 10.22331/q-2024-05-14-1344.

[28] A. M. Stephens, "Fault-tolerant thresholds for quantum error correction with the surface code," Phys. Rev. A, vol. 89, no. 2, p. 022321, Feb. 2014, doi: 10.1103/PhysRevA.89.022321.

[29] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," Phys. Rev. A, vol. 86, no. 3, p. 032324, Sep. 2012, doi: 10.1103/PhysRevA.86.032324.

[30] L. Voss, J. X. Sim, T. Haug, and K. Bharti, "Multivariate bicycle codes," Phys. Rev. A, vol. 111, no. 6, p. L060401, Jun. 2025, doi: 10.1103/ll5p-z88p.

[31] A. G. Fowler, A. M. Stephens, and P. Groszkowski, "High-threshold universal quantum computation on the surface code," Phys. Rev. A, vol. 80, no. 5, p. 052312, Nov. 2009, doi: 10.1103/PhysRevA.80.052312.

[32] IBM Quantum, "IBM quantum delivers on 2022 100×100 performance challenge," IBM Quantum Blog, Nov. 2024, accessed: Oct. 2, 2026. [Online]. Available: https://www.ibm.com/quantum/blog/qdc-2024

[33] C. Gidney and M. Ekerå, "How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits," Quantum, vol. 5, p. 433, Apr. 2021, doi: 10.22331/q-2021-04-15-433.

[34] C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits," 2025, arXiv:2505.15917.

[35] J. Lee et al., "Even more efficient quantum computations of chemistry through tensor hypercontraction," PRX Quantum, vol. 2, p. 030305, Jul. 2021, doi: 10.1103/PRXQuantum.2.030305.

[36] R. Babbush, J. R. McClean, M. Newman, C. Gidney, S. Boixo, and H. Neven, "Focus beyond quadratic speedups for error-corrected quantum advantage," PRX Quantum, vol. 2, p. 010103, Mar. 2021, doi: 10.1103/PRXQuantum.2.010103.

[37] M. H. Shaw and B. M. Terhal, "Lowering connectivity requirements for bivariate bicycle codes using morphing circuits," Phys. Rev. Lett., vol. 134, no. 9, p. 090602, Mar. 2025, doi: 10.1103/PhysRevLett.134.090602.

[38] G. Torlai and R. G. Melko, "Neural decoder for topological codes," Phys. Rev. Lett., vol. 119, no. 3, p. 030501, Jul. 2017, doi: 10.1103/PhysRevLett.119.030501.

[39] J. Bausch, A. W. Senior, F. J. H. Heras et al., "Learning high-accuracy error decoding for quantum processors," Nature, vol. 635, pp. 834–840, Nov. 2024, doi: 10.1038/s41586-024-08148-8.

[40] Google Quantum AI and Collaborators, "Quantum error correction below the surface code threshold," Nature, vol. 638, pp. 920–926, Feb. 2025, doi: 10.1038/s41586-024-08449-y.

[41] L. Caune, L. Skoric, N. S. Blunt et al., "Demonstrating real-time and low-latency quantum error correction with superconducting qubits," Nat. Commun., vol. 17, no. 1, p. 7383, Jun. 2026, doi: 10.1038/s41467-026-73331-6.

[42] A. Cicero, L. Altamura, M. Lange, M. Granath, and P. Trancoso, "Low latency GNN accelerator for quantum error correction," 2026, arXiv:2603.22149.

[43] G. Aleksandrowicz et al., "Qiskit: An open-source framework for quantum computing," 2019, doi: 10.5281/zenodo.2562111. [Online]. Available: https://qiskit.org

[44] J. C. Spall, "Multivariate stochastic approximation using a simultaneous perturbation gradient approximation," IEEE Trans. Autom. Control, vol. 37, no. 3, pp. 332–341, Mar. 1992, doi: 10.1109/9.119632.